Enterprise Agentic RAG Platform

End-to-End System Architecture, Code Design, LangGraph Workflow & Azure Deployment Guide

1. Executive Summary & Problem Statement

This project implements an enterprise-grade, multi-agent conversational AI platform capable of retrieving answers from a knowledge base containing **100,000+ unstructured documents** (PDF, DOCX, PPTX, TXT, HTML) and performing **real-time web search** via the Model Context Protocol (MCP). The entire architecture is built with Clean Architecture, LangGraph, FastAPI, Streamlit, and Azure Cloud Services.

Core System Principles:

- **Strict Minimalist UX:** ChatGPT / Copilot style interface with zero administrative clutter.
- **Zero-Cost Local Embeddings:** CPU ONNX FastEmbed engine avoiding API costs and rate limits.
- **Serverless Cloud Infrastructure:** Scaled-to-Zero ($0 idle cost) on Azure Container Apps and Azure Cosmos DB Free Tier.
- **Verified Grounding:** Real-time bracket citations ([1], [2]) verified by an automated citation guardrails agent.

2. System Architecture & Components

Component	Technology	Key Role & Configuration
Frontend UI	Streamlit 1.39+	Minimalist chat UI, thread switching, citation expanders, feedback (port 8501).
Backend API	FastAPI (Async First)	Asynchronous REST API, CORS middleware, dependency injection (port 8000).
Agent Framework	LangGraph + LangChain	State machine orchestrating Planner, Retrieval, MCP Tool, Response, and Citation nodes.
Vector Search	Azure AI Search (Free Tier)	Hybrid dense vector search (HNSW) combined with BM25 keyword matching.
Embeddings Engine	FastEmbed (ONNX)	BAAI/bge-small-en-v1.5 running locally on CPU (384-dimensional dense vectors).
Tool Integration	MCP (Model Context Protocol)	Streamable HTTP client connecting to hosted Tavily MCP for live web news & search.
Session Memory	Azure Cosmos DB (Free Tier)	Document store partitioned by /userId for sub-5ms history lookups.
Foundation Models	Azure AI Foundry	Llama-3.3-70B for intent planning & Phi-4 for grounded response generation.
Container Hosting	Azure Container Apps	Serverless Kubernetes with KEDA autoscaling, scale-to-zero, and automated HTTPS.

3. LangGraph Agentic Workflow & Reasoning Pipeline

The reasoning process is modeled as an asynchronous **StateGraph** with dynamic conditional routing:

1. Planner Agent: Analyzes the user question and the last 6 turns of conversation history. Produces a strict structured plan: {use_documents: bool, use_web: bool}.

2. Retrieval Agent: Invoked when use_documents == True. Embeds query into 384-dim vector and runs hybrid BM25 + vector search against Azure AI Search.

3. MCP Tool Agent: Invoked when use_web == True. Connects to Tavily Search via MCP. Detects topic categories ('news' vs 'general') and extracts high-relevance snippets.

4. Response Synthesis Agent: Compiles all retrieved sources into a numbered reference block ([1], [2], [3]) and instructs the model to answer strictly using the provided context.

5. Citation & Guardrails Agent: Validates all [n] brackets in the generated text, strips dangling hallucinations, and filters the metadata citations list to match only sources cited in the final answer.

4. How Azure Container Apps (ACA) Works & Scales

Azure Container Apps is a serverless platform built on top of managed Kubernetes (AKS), Envoy Proxy, and KEDA (Kubernetes Event-driven Autoscaler).

Key Architectural Capabilities:

• **Scale to Zero (minReplicas: 0):** When no requests arrive, ACA shuts down all running replicas. The application consumes 0 CPU and 0 RAM, costing exactly **$0.00** while idle.

• **KEDA Dynamic Scaling:** When traffic spikes (e.g. 50 concurrent requests per container), KEDA automatically spins up new replicas in seconds up to maxReplicas.

• **Envoy Ingress & Free HTTPS:** Automatic SSL certificate provisioning, internal/external ingress routing, and health checks on /health and /_stcore/health.

• **Microservice Decoupling:** The Streamlit frontend and FastAPI backend scale independently according to their specific load profiles.

5. Key Production Issues & Engineering Solutions

Issue Diagnosed	Root Cause	Production Resolution Applied
Cosmos DB 401 on Swagger UI	Default Swagger template sent dummy conversation_id = 'string'.	Updated ChatService to defensively treat non-UUID / empty IDs as fresh conversations.
Tavily Returning Irrelevant Code	Default basic search depth without topic routing matched shallow git repos.	Added dynamic keyword routing in tavily.py: queries with 'news/market' use topic='news' & depth='advanced'.
Docker Permission Denied on Cache	Non-root container user (appuser) lacked write access to /app/.cache.	Configured HF_HOME and FASTEMBED_CACHE_PATH to /tmp/cache with full chown permissions in Dockerfile.
Foundry 404 Endpoint Duplication	OpenAI client SDK appends /chat/completions to base endpoint url.	Corrected AZURE_AI_FOUNDRY_CHAT_ENDPOINT to https://.services.ai.azure.com/models.

6. Production Deployment Configuration

Streamlit Frontend: Azure Container App (Port 8501, Ingress: External)

FastAPI Backend: Azure Container App (Port 8000, Ingress: External)

Deployment Platform: Azure Container Apps (Scale-to-Zero Enabled)

GitHub Repository: https://github.com/tadevilas/End-to-End-AgenticBot-Azure-